CSCI 3388 Practice Midterm **SOLUTIONS**

Instructions:

- **DO NOT DETACH ANY PAGES—EVEN THE SCRATCH PAGES—FROM THIS EXAM!** Removing page(s) may result in a deduction in your grade.
- This exam is closed book and closed notebook, with no electronic devices allowed. The only resource you may use is one 8.5×11-inch double-sided study sheet that you prepared yourself.
- You are not allowed to discuss the exam with anyone until the solutions are published.

Any deviation from these rules will constitute a violation of the University policies on academic integrity.

- The exam is designed for 75 minutes and consists of multiple-choice and written-answer problems.
- For the multiple-choice problems, mark your answers by filling in the bubble to the left of your choice. Select only one choice.
- For the written-answer problems, write your answers clearly in the provided space.
- The exam has 6 pages printed on both sides, including this page and three scratch pages at the end.

Name: _____

Email: _____

Eagle ID: _____

Section	Points
1-4 (Multiple Choice)	/ 20
5-8 (Written Answer)	/ 80
Total	/ 100

Multiple Choice (5 points each)

1. N-gram Language Models

   Given the following bigram counts from a corpus:

   - count("the", "cat") = 15
   - count("the") = 100
   - count("cat") = 20

   What is P("cat" | "the") using maximum likelihood estimation?

   ◯ 0.15
   ● **0.20**
   ◯ 0.75
   ◯ 0.80

   Solution: (A). P("cat"|"the") = count("the","cat")/count("the") = 15/100 = 0.15

2. Neural Network Activation Functions

   Which activation function is most commonly used in modern deep learning to address the vanishing gradient problem?

   ◯ Sigmoid
   ◯ Tanh
   ● **ReLU**
   ◯ Linear

   Solution: (C). ReLU addresses the vanishing gradient problem by having a constant gradient of 1 for positive inputs.

3. Attention Mechanism

   In the scaled dot-product attention formula Attention(Q,K,V) = softmax($QK^T/\sqrt{d_k}$)V, what is the purpose of the scaling factor $\sqrt{d_k}$?

   ◯ To normalize the attention weights
   ● **To prevent the dot products from becoming too large**
   ◯ To ensure the softmax outputs sum to 1
   ◯ To make the computation more efficient

   Solution: (B). The scaling factor prevents the dot products from becoming too large, which would cause the softmax to saturate.

4. GPT Evolution

   Which GPT model first demonstrated significant zero-shot capabilities without task-specific fine-tuning?

   ◯ GPT-1

● **GPT-2**
○ GPT-3
○ ChatGPT

Solution: (B). GPT-2 was the first to demonstrate significant zero-shot capabilities through scale and prompting.

Written Answer (80 points total)

5. Attention Scaling (20 points)

   a) Derive why variance of dot product $q \cdot k$ grows with d_k if $q, k \sim \mathcal{N}(0, 1)$.

   b) Show dividing by $\sqrt{d_k}$ stabilizes variance.

   c) What happens to attention weights when d_k becomes very large without scaling? Explain the practical implications for training transformers.

Solution:

a) If $q, k \sim \mathcal{N}(0, I_{d_k})$, then $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$.

Since q_i, k_i are independent standard normal variables, $q_i k_i$ has mean 0 and variance $\text{Var}(q_i) \cdot \text{Var}(k_i) = 1 \cdot 1 = 1$.

Therefore: $\text{Var}(q \cdot k) = \text{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k$

b) Scaling by $\frac{1}{\sqrt{d_k}}$:

$$\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{1}{d_k} \cdot \text{Var}(q \cdot k) = \frac{1}{d_k} \cdot d_k = 1$$

c) When d_k becomes very large without scaling:

- **Attention weights become uniform**: As $d_k \to \infty$, the variance of $q \cdot k$ grows without bound, making all dot products very large
- **Loss of selectivity**: The model can no longer focus on specific relevant tokens, losing the ability to learn meaningful attention patterns

Practical implications for training:

- **Gradient problems**: Uniform attention weights lead to vanishing gradients, making training extremely slow or impossible
- **Training instability**: Large, unnormalized dot products can cause numerical instability and overflow issues

This is why the $\sqrt{d_k}$ scaling is crucial for transformer training - it maintains the model's ability to learn meaningful attention patterns regardless of the embedding dimension.

6. Transformer Architecture and Multi-Head Attention (20 points)

You're implementing a transformer model and need to understand the attention mechanism in detail.

   a) (7 pts) Explain how multi-head attention works. What are the benefits of having multiple attention heads?

   b) (7 pts) Given query Q, key K, and value V matrices of shape (seq_len, d_model), derive the computational complexity of self-attention.

   c) (6 pts) How would you modify the attention mechanism to handle very long sequences (10k+ tokens) efficiently?

Solution:

a) **Multi-head attention:**

- **Process**: Split d_model into h heads, each with dimension d_k = d_model/h
- **Computation**: Each head computes attention independently: $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$
- **Concatenation**: $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$
- **Benefits**: Captures different types of relationships (syntax, semantics, long-range), parallelizable, more expressive than single head

b) **Computational complexity:**

- **Attention weights**: QK^T has shape (seq_len, seq_len), complexity $O(seq\_len^2 \cdot d\_model)$
- **Softmax**: $O(seq\_len^2)$ for each head
- **Value multiplication**: $O(seq\_len^2 \cdot d\_model)$
- **Total**: $O(seq\_len^2 \cdot d\_model)$ - quadratic in sequence length

c) **Efficient attention for long sequences:**

- **Sparse attention**: Only attend to nearby tokens or selected positions
- **Sliding window**: Limit attention to a fixed window size
- **Hierarchical attention**: Attend at multiple granularities (sentence, paragraph, document)
- **Linear attention**: Use approximations like Performer or Linformer
- **Memory-efficient**: Gradient checkpointing, recompute attention weights

7. Value Iteration in an MDP (20 points)

Consider a 3-state MDP: (s_1, s_2, s_3) with deterministic transitions and rewards:

- s_1: A $\to s_1$, reward 1; B $\to s_2$, reward 0.
- s_2: A $\to s_2$, reward 2; B $\to s_3$, reward 0.
- s_3: any action $\to s_3$, reward 5.

Discount $\gamma = 0.9$.

a) (8 pts) Write the Bellman optimality equation.

b) (8 pts) Starting with $V_0(s) = 0$, perform one value iteration update. Compute $V_1(s_1), V_1(s_2), V_1(s_3)$.

c) (4 pts) Which action is preferred in s_1 after one iteration?

Solution:

(a) The Bellman optimality equation is:

$$V^*(s) = \max_a \{R(s, a) + \gamma V^*(s')\}$$

where s' is the next state after taking action a in state s.

(b) Starting with $V_0(s) = 0$ for all states, one value iteration update:

For s_1:

- Action A: $R(s_1, A) + \gamma V_0(s_1) = 1 + 0.9 \cdot 0 = 1$
- Action B: $R(s_1, B) + \gamma V_0(s_2) = 0 + 0.9 \cdot 0 = 0$
- $V_1(s_1) = \max(1, 0) = 1$

For s_2:

- Action A: $R(s_2, A) + \gamma V_0(s_2) = 2 + 0.9 \cdot 0 = 2$
- Action B: $R(s_2, B) + \gamma V_0(s_3) = 0 + 0.9 \cdot 0 = 0$
- $V_1(s_2) = \max(2, 0) = 2$

For s_3:

- Any action: $R(s_3, \cdot) + \gamma V_0(s_3) = 5 + 0.9 \cdot 0 = 5$
- $V_1(s_3) = 5$

(c) At s_1 after one iteration, action A is preferred because it gives value 1 vs action B's value 0.

8. GPT Alignment & Safety (20 points)

Your LLM is generating harmful content and you need to implement RLHF to fix this.

a) (10 pts) How would you design the human preference data collection process for RLHF?

b) (10 pts) Your model is reward hacking (optimizing for the reward model instead of being helpful). How would you detect and fix this?

Solution:

a) **Data Collection Process:**

- **Prompt design**: Cover helpfulness, safety, factuality across domains and difficulty levels
- **Response generation**: Sample diverse responses using different temperatures/models
- **Annotation**: Pairwise preferences with 5-point rubrics (Helpful, Honest, Harmless)
- **Quality control**: Inter-annotator agreement ($\alpha \geq 0.6$), rater calibration, adversarial checks
- **Sampling**: Active learning on uncertain/disagreed cases, upweight failure modes
- **Bias mitigation**: Track subgroup performance, use counterfactual prompts

b) **Reward Hacking Detection & Fix:**

- **Detection**: Monitor RM-human gap, metric drift (length/repetition), ensemble disagreement, stress tests
- **Data fixes**: Add hard negatives, mine adversarial examples, include absolute ratings
- **RM fixes**: Calibrate/regularize, use ensemble/pessimistic RM, multi-objective optimization
- **RL fixes**: Increase KL penalty, add exploit penalties, use constrained RL
- **Evaluation**: A/B testing with human judgment, frozen challenge sets, canary monitoring